Blitz Planning -> Azure: inventaris & migratieplan

Opgesteld 2026-10-16. Bedoeld om samen met de IT-/Azure-beheerder van het werk door te nemen.

*Huidige situatie: de app draait volledig op **Netlify** (gratis/pro hosting + serverless). Doel: alles overzetten naar de **Azure-omgeving van het werk**.*

---

1. Wat de planner allemaal nodig heeft (volledige oplijsting)

1.1 Code & repository

Wat	Waar nu
Broncode	GitHub, privé-repo `Brekke20/Blitz-Panning`
Grootte	~2 MB frontend + ~1 MB backend, repo-historiek ~93 MB
Versie	v2.1.6
Opbouw	Eén webpagina-app (PWA) + ~50 kleine backend-functies

1.2 De twee delen van de app

1. **De website zelf (frontend)** — de map `public/`. Dit is "statische" inhoud: één `index.html`, een hoop JavaScript-modules, CSS en een manifest. Er is **geen build-stap** nodig (geen compilatie); de bestanden worden zo geserveerd. Werkt ook offline dankzij een service worker.
2. **De backend (serverloze functies)** — de map `netlify/functions/`, ~50 losse functies (o.a. inloggen, tickets ophalen, plannen, voorstel sturen, annuleren, route berekenen, rapport maken, dashboard). Ze worden aangesproken via adressen die op `/api/...` beginnen.

1.3 Data-opslag

- Alle gegevens (planning, gebruikers, instellingen, afspraken, rapport-register, spel-scores, activiteitenlog …) staan in **Netlify Blobs**, één opslag genaamd `blitz-data`, met "strong consistency" (= je leest altijd meteen de laatst geschreven waarde).
- Technisch is dit een **sleutel-waarde-opslag** (per sleutel een stukje JSON). Geen klassieke SQL-database.
- Er is **geen** externe database, geen Postgres/MySQL. Dit is belangrijk: bij een verhuis moet deze opslag mee of vervangen worden.

1.4 Externe koppelingen (blijven ongewijzigd, draaien buiten Azure)

Koppeling	Waarvoor	Hoe
Zoho Desk	Tickets ophalen, voorstel-/annuleer-mails naar de klant sturen, rapport-mails	API met een vaste "refresh token" (OAuth)
TomTom	Reistijden, routes, verkeersdrukte, geocoderen van adressen	API-sleutel

Deze diensten blijven gewoon bestaan; enkel de app die ze aanroept verhuist.

1.5 Geheimen & instellingen (de "env vars" — mee te verhuizen)

Dit zijn de wachtwoorden/sleutels die de backend nodig heeft. Ze staan **niet** in de code maar als instelling bij de hostingprovider. Bij Azure moeten ze opnieuw ingesteld worden (liefst in een Key Vault):

Instelling	Waarvoor
`ZOHO_CLIENT_ID`, `ZOHO_CLIENT_SECRET`, `ZOHO_REFRESH_TOKEN`	Toegang tot Zoho Desk
`ZOHO_FROM_EMAIL`	Afzender van de klantmails
`TOMTOM_API_KEY`	Toegang tot TomTom
`SESSIE_GEHEIM`	Ondertekent de login-sessie (cookie)
`CONFIRM_LINK_SECRET`	Beveiligt de bevestigingslink in afspraak-mails
`PLANNING_EXPORT_API_KEY`	Sleutel voor de externe planning-export

Instelling	Waarvoor
`URL`	Het eigen webadres van de app (nu automatisch door Netlify geleverd — op Azure zelf instellen)
`BLITZ_VANGNET_ZELF`	Schakelaar voor de "vangnet"-controle

1.6 Externe bibliotheken (worden van internet geladen, niet mee verhuisd)

De app haalt enkele hulpbibliotheken rechtstreeks van publieke CDN's. Die blijven werken, maar het is goed te weten dat de app die hosts nodig heeft:

- **Leaflet 1.9.4** (kaart) — van `cdnjs.cloudflare.com`
- **signature_pad 4.1.7** (handtekening) — van `cdnjs.cloudflare.com`
- **ExcelJS 4.4.0** (Excel-export) — van `cdn.jsdelivr.net`
- **Google Fonts** — van `fonts.googleapis.com` / `fonts.gstatic.com`

1.7 Drie "speciale" backend-stukken (de aandachtspunten voor de verhuis)

1. **PDF-rapporten maken** — de functie `rapport` start een volledige (headless) **Chrome-browser** in de cloud om een PDF te genereren (via `puppeteer-core` + `@sparticuz/chromium-min`). Dit is specifiek gebouwd voor de Netlify/AWS-manier van werken en is **het moeilijkste stuk** om naar Azure te brengen.
2. **Een geplande taak** — `rapport-vangnet` draait **automatisch elke 5 minuten** (een cron-taak) als vangnet voor rapport-mails.
3. **Een achtergrondtaak** — `rapport-verwerk-background` verwerkt een rapport los van het verzoek (Netlify stuurt meteen "ontvangen" terug en werkt daarna verder). Azure doet dit anders (via een wachtrij/timer).

Verder hebben verschillende functies een **langere time-out** nodig (tot 26–30 seconden i.p.v. de standaard ~10 s): rapport, send-rapport, propose, annuleer, dashboard, mail-check, planning-export, enz.

---

2. Hoe krijgen we dit op Azure?

2.1 Aanbevolen opzet

De meest natuurlijke tegenhanger van Netlify binnen Azure is **Azure Static Web Apps**. Dat combineert in één dienst: de statische website hosten + de backend-functies + de `/api`-routering + gratis SSL + een kant-en-klare koppeling met GitHub. Daarnaast één opslag voor de data en, voor de PDF's, een apart stukje.

Nu op Netlify	Straks op Azure
Statische site (`public/`)	**Azure Static Web Apps** (frontend)
Serverless functies (`netlify/functions/`)	**Azure Functions** (automatisch gekoppeld aan de Static Web App)
`/api/*`-routering (redirect)	Ingebouwd in Static Web Apps (+ `staticwebapp.config.json`)
Netlify Blobs (`blitz-data`)	**Azure Blob Storage** (zelfde idee: sleutel-waarde-JSON; ondersteunt óók de optimistic-locking die de code wil, via ETags)
Geplande taak (cron)	**Azure Functions Timer-trigger**
Achtergrondtaak	**Azure Functions** met een **Queue-trigger** (Azure Storage Queue)
PDF via Chromium	**Apart**: Azure Container Apps of een container-gebaseerde Azure Function met Playwright (zie 3.1)
Geheimen (env vars)	**Azure Key Vault** + app-instellingen
Auto-deploy bij push	**GitHub Actions** (Static Web Apps levert dit standaard)

2.2 Grof stappenplan

1. **Toegang & eigenaarschap regelen**: onder welke Azure-subscription van het werk, wie beheert ze, wie betaalt, welke "resource group".
2. **Static Web App aanmaken** en koppelen aan de GitHub-repo -> de frontend draait.
3. **Functies overzetten** naar het Azure Functions-formaat (de code is grotendeels herbruikbaar, maar de "verpakking" rond elke functie verschilt).

4. **Data-opslag vervangen**: Netlify Blobs -> Azure Blob Storage. Dit is één gedeeld stukje code dat overal gebruikt wordt, dus centraal aan te passen. **Bestaande data migreren** (exporteren uit Netlify, importeren in Azure).
5. **Geheimen opnieuw instellen** in Azure (Key Vault).
6. **PDF-generatie apart oplossen** (zie 3.1).
7. **Geplande + achtergrondtaak** omzetten naar Azure-triggers.
8. **Domein & SSL**: eigen webadres laten wijzen naar Azure, Zoho-bevestigingslinks en afzender controleren.
9. **Testen op een testomgeving**, daarna pas overschakelen.

---

3. Waar we op moeten letten (risico's & valkuilen)

1. **PDF-rapporten (grootste risico).** De huidige PDF-aanpak (`@sparticuz/chromium-min`) is specifiek voor AWS/Netlify. Op Azure moet dit herbouwd worden, meestal met **Playwright in een container**. Dit vraagt het meeste werk en testen. Niet onderschatten.
2. **Data-migratie, niet enkel de code.** Alle planning, gebruikers en instellingen zitten in Netlify Blobs. Die data moet mee verhuizen en na de overschakeling **kloppen en compleet zijn**. Plan een exacte kopie + controle.
3. **Niets mag dubbel draaien tijdens de overgang.** Zolang Zoho-mails verstuurd worden, mogen de oude (Netlify) en nieuwe (Azure) versie niet tegelijk live zijn — anders riskeer je dubbele mails naar klanten. Eén duidelijk omschakelmoment.
4. **Time-outs.** Azure's standaardplan ("Consumption") beperkt hoe lang een functie mag draaien. Verschillende functies hebben 26–30 s nodig. Mogelijk is een **hoger plan** (Premium) nodig, met kostengevolg.
5. **"Strong consistency".** De app rekent erop dat ze meteen de laatst geschreven waarde terugleest. Azure Blob Storage kan dit, maar de manier van "veilig overschrijven" (nu een eigen truc in de code) moet zorgvuldig omgezet worden naar de Azure-manier (ETags), anders riskeer je verloren wijzigingen bij twee mensen tegelijk.
6. **De "URL"-instelling.** Netlify vult het eigen webadres automatisch in; dat gebruikt de app o.a. in de klant-links. Op Azure moet dit expliciet en correct gezet worden.
7. **Geheimen veilig overzetten.** De Zoho- en TomTom-sleutels zijn gevoelig. Niet via mail/chat doorsturen; de IT-beheerder zet ze rechtstreeks in Azure Key Vault.
8. **De repo staat op een persoonlijke GitHub-account** (`Brekke20`). Voor een werk-Azure wil IT dit misschien onder een werk-/organisatie-account of aparte deploy-rechten. Afspreken.
9. **Service worker / cache.** De app bewaart zichzelf lokaal in de browser (offline-werking). Bij de overschakeling moet de cache-versie opgehoogd worden, anders blijven gebruikers op de oude versie hangen.
10. **De externe CDN's** (Leaflet, ExcelJS, Google Fonts) moeten bereikbaar blijven. Als het bedrijfsnetwerk bepaalde externe hosts blokkeert, werkt de kaart of de export niet. Eventueel die bibliotheken mee hosten.
11. **Kosten.** Netlify is nu (groten)deels gratis. Azure rekent per gebruik (opslag, functie-uitvoeringen, eventueel Premium-plan en de PDF-container). Vooraf een inschatting vragen aan IT.

---

4. Wat we best doen om alles vlot te laten verlopen

1. **Eerst een kleine proef, niet alles ineens.** Eén testomgeving op Azure met de frontend + een paar eenvoudige functies + een test-opslag. Zo weten we hoe Azure zich gedraagt vóór we de echte data aanraken.
2. **De data-opslag als één centraal stuk aanpakken.** Alle functies gaan via hetzelfde kleine hulpstukje naar de opslag. Als we dát ene stuk netjes van "Netlify Blobs" naar "Azure Blob Storage" omzetten, volgt de rest vanzelf. Dit eerst doen en apart testen.
3. **De PDF-generatie als los deelproject.** Het moeilijkste stuk vroeg vastpakken en apart bewijzen dat het werkt, los van de rest. Niet tot het einde bewaren.
4. **Automatische tests mee laten lopen.** De app heeft een uitgebreide testset (honderden tests). Die helpen bewijzen dat de Azure-versie zich hetzelfde gedraagt als nu. Laten draaien na elke stap.
5. **De IT-/Azure-beheerder vroeg betrekken** voor: subscription, rechten, Key Vault, kosteninschatting, netwerkregels (externe CDN's), en het domein/DNS. Veel hiervan beslist IT, niet wij.
6. **Een draaiboek voor het omschakelmoment.** Op voorhand vastleggen: wanneer, wie zet Netlify stil, hoe controleren we dat Azure werkt, en een "terugvalplan" als er iets misloopt (Netlify tijdelijk terug aanzetten).
7. **Niets live zetten zonder Brents akkoord**, zoals altijd — zeker het moment waarop de klantmails via Azure gaan lopen.

Samengevat voor de beslissing

- **Kan het?** Ja. Azure Static Web Apps + Functions + Blob Storage is een goede match voor deze app.
- **Grootste werk/risico:** de PDF-rapporten (Chrome in de cloud) en het correct mee-verhuizen van alle data.
- **Eerst te beslissen door het werk/IT:** welke Azure-subscription, budget, rechten op de GitHub-repo, en of het Consumption- of Premium-plan nodig is.